Working with Claude End-to-End: From Jira Ticket to Merged PR

Oct 29, 2026 · @anat

Purpose and roles

Designers use Claude to take a Jira ticket all the way to a tested, self-reviewed PR; the developer reviews, approves and merges. These notes come from a session with a senior frontend developer on how to set that up.

- **Designers (3):** write clear tickets, run Claude through the workflow below, and open PRs that already pass self-review and tests.
- **Developer (code owner):** owns the guidelines and skills, reviews every PR, and is the only one who merges.
- **Claude:** reads the ticket, plans, implements, reviews its own work against our guidelines, and writes the tests.

The goal: PRs arrive in a predictable shape, so review is about the change, not about style or missing tests.

The workflow, end to end

Every task moves through the same nine stages. Designers own stages 1–5; the developer owns review and release.

1. **To do** — the Jira ticket is written and ready (see the next section).
2. **Gathering data** — Claude reads the ticket, the files it points to, and example files.
3. **Plan** — Claude proposes a plan; the designer checks it before any code is written.
4. **Implementation** — Claude writes the code and the tests, using our skills.
5. **Self-review** — the reviewer skill and pre-commit checks run; issues are fixed before the PR opens.
6. **Team review** — the developer reviews the PR and approves or requests changes.
7. **CI/CD** — the pipeline runs, including Chromatic visual tests.
8. **Production review** — check the change live, behind its feature flag if there is one.
9. **Done** — only when the definition of done is met.

Helping Claude help you: the Jira ticket

Claude reads the ticket directly from Jira, so the ticket is the prompt. A vague ticket gives a vague PR. Every ticket should include:

Field	What to write
Description	What is changing and why, in a few sentences
Content	Exact copy, labels, states, links to designs
Code area	Where in the code this lives: the starting file, plus 2–3 existing files to copy patterns from
Definition of done	What must be true for the task to be finished (see below)
Related PRs / files	Earlier work on the same feature
Feature flag	Whether the change sits behind a flag, and which one
Special cases	Edge cases, empty states, errors, permissions, RTL

The most useful line is the explicit pointer to code: "start in file X, follow the pattern in files 1, 2 and 3."

Guidelines for Claude

Claude follows what is written down, so our standards live in markdown files in the repo, owned by the developer.

- **CLAUDE.md (and the Cursor rules file, if anyone uses Cursor):** the entry point Claude reads at the start of every session. It links to the detailed guideline files.
- **What the guidelines cover:**
  - How we write CSS (naming, structure, tokens, what to avoid)
  - Good practice vs bad practice, with short code examples of each
  - The Angular version we are on, so Claude doesn't use newer or older APIs
  - Patterns for Claude: the recurring structures we want copied
- **/context:** run it to see what Claude has loaded and how it indexed the codebase. Use it when Claude seems to miss something obvious.

- **Learn from strong reviewers:** go through PR comments from developers we respect and turn their recurring feedback into guideline rules.

Where to collect best practices

Take Angular syntax and patterns from the official files and the core team, and take component, CSS, accessibility and Storybook practices from design system maintainers. Store every rule in the repo (for example `docs/guidelines/angular.md`, `css.md`, `testing.md`), linked from CLAUDE.md, as a do/avoid pair with a one-line why and a short example.

Official sources (start here)

- Angular best-practices.md for LLMs: the Angular team's own rules file for AI tools; use it as the base of CLAUDE.md and check it against our Angular version.
- Angular style guide: naming, `inject()`, readonly inputs and outputs, class and style bindings.
- Web Codegen Scorer (same page): the Angular team's tool for scoring AI-generated code; useful for our evals.

Design system maintainers (most relevant for the designers)

Who	GitHub	Repo	Mine for
Andrew Seguin	`andrewseguin`	angular/components	Theming, core styles
Jeremy Elbourn	`jelbourn`	angular/components	Theming, CDK accessibility
Miles Malerba	`mmalerba`	angular/components	Design tokens
Paul Gschwendtner	`devversion`	angular/components	CDK accessibility
Robin Götz	`goetzrobin`	spartan-ng/spartan	Headless accessible primitives; the repo ships an AI agent skill to copy from
Alex Inkin	`waterplea`	taiga-family/taiga-ui	Component architecture; has a Figma library
Clarity team	—	vmware-clarity/ng-clarity	Enterprise design system: tokens, theming at scale

Who	GitHub	Repo	Mine for
SAP team	—	SAP/fundamental-ngx	Enterprise design system

Angular core and ecosystem (for Angular idioms)

Who	GitHub	Focus
Kristiyan Kostadinov	`crisbeto`	Framework and components; very active reviewer
Matthieu Riegler	`JeanMeche`	Reviews and docs; comments explain the why
Andrew Scott	`atscott`	Angular team; merges framework PRs
Alex Rickabaugh	`alxhub`	Angular team; merges framework PRs
Andrew Kushnir	`AndrewKushnir`	Angular team; merges framework PRs
Jessica Janiuk	`thePunderWoman`	Angular core team
Brandon Roberts, Marko Stanimirović	`brandonroberts`, `markostanimirovic`	NgRx: state and signals
Rainer Hahnekamp, Manfred Steyer, Enea Jahollari, Netanel Basal	—	Architecture and testing content

Roles change: check each person's recent activity, and the current angular/components CODEOWNERS file, before mining.

How to mine them

1. Use the `gh` CLI to pull one reviewer's recent review comments on one repo.
2. Have Claude group them into themes and draft do/avoid rules.
3. Keep only rules that fit our codebase and Angular version.
4. Do the same with our own PR history; repeated comments become lint rules or guideline lines.

Skills to build

A skill is a reusable instruction set Claude runs on demand. These are the ones we need; existing examples can be used as a starting point.

Skill	What it does	When it runs
Reviewer	Runs 3 review agents one after another, each checking the code against the guidelines. If one finds a problem, Claude stops, fixes it, then moves to the next agent.	Self-review, before opening a PR
Pre-commit	Runs the reviewer agent automatically	On every commit
Repeated code	For code we write again and again: knows which files to read and already holds the prompt, so Claude adds it the same way each time	During implementation
Accessibility	Checks and fixes accessibility issues	During implementation and review
Test builder	Writes the integration and regression tests (next section)	Part of the definition of done
Orchestration	Runs codemods for strict, mechanical changes, and handles the open-ended cases where the result isn't known up front	Large or repeated refactors
Build PR	Assembles the PR: title, description, linked ticket	End of implementation
Split PR	Breaks a large PR into smaller, reviewable ones	When a PR grows too big

Each skill should state its own definition of done, so Claude knows when its part is finished.

Testing

Every PR ships with two kinds of tests, written by the test-builder skill.

- **Integration tests:** test the feature from the point the backend responds through to the final UI state, assuming the backend returns the expected data.
- **Regression tests (Storybook):**
  - **Chromatic** takes snapshots before and after the change and flags any visual difference.
  - **Storybook play functions** script user interactions (clicks, typing) and check the result.

A **Chromatic MCP** connection lets Claude read Chromatic results directly and fix visual regressions itself.

Definition of done and evals

A task is done only when an automated check says so, not when it looks finished. That check is the **eval**: something that runs, and only when it passes is the task complete.

A PR is ready for review when:

- [ ] The ticket's own definition of done is met
- [ ] The reviewer skill ran and every issue was fixed
- [ ] Pre-commit checks pass
- [ ] Integration tests are added and pass
- [ ] Storybook stories and play functions are added; Chromatic shows only intended changes
- [ ] Accessibility check passes
- [ ] The PR description links the ticket and notes any feature flag

Keeping Claude's context clean

Claude works worse when its context fills up with noise. Use a tool or script that regularly clears stale content and cache from the context, and start a fresh session for each new task.

Next steps

- [ ] Write CLAUDE.md and the first guideline files (CSS, good vs bad, Angular version)
- [ ] Collect best practices from senior developers' PR comments

- [ ] Build the reviewer and pre-commit skills first, then the test builder
- [ ] Set up Chromatic and the Chromatic MCP
- [ ] Agree on the Jira ticket template with the designers
- [ ] Run one real ticket through the full workflow together

Sources

- Develop with AI – angular.dev
- Angular style guide
- angular/components CODEOWNERS
- spartan-ng/spartan
- Ng-News 26/16: spartan/ui 1.0
- Alexander Inkin – Google Dev Library
- Angular UI libraries – Trendshift
- Marko Stanimirović – Angular Architects